

Лабораторная работа №35. Подключаем фронтенд к API: доска мемов

Цель работы:

- Понять, как браузерный JavaScript общается с сервером через fetch.
- Разобраться, почему браузер блокирует запросы между разными портами и как это исправить.
- Построить страницу с нуля: HTML + CSS + JS в отдельных файлах.
- Реализовать полный цикл: загрузить список, добавить, удалить — всё через fetch.

Краткая теория (вводная часть)

Как фронтенд общается с бэкендом

До этой лабораторной мы работали с API через curl и браузер. Теперь сделаем так, чтобы **наша собственная HTML-страница** обращалась к серверу и отображала данные.

Схема работы:

Браузер открывает **index.html**

↓

JavaScript делает **fetch('http://localhost:5000/api/memes')**

↓

Сервер (ASP.NET Core) обрабатывает **запрос**

↓

Возвращает JSON: [{"id":1,"title":"..."},...]

↓

JavaScript получает данные и **отрисовывает** карточки на странице

Всё это происходит **без перезагрузки страницы** — пользователь добавляет мем, карточка появляется сразу.

Почему браузер блокирует запросы — CORS

Есть важный момент: браузер не разрешает JavaScript обращаться к серверу на **другом порту**. Наша страница открыта на порту 5500 (Live Server в VS Code), а API работает на порту 5000. Это разные адреса с точки зрения браузера — он заблокирует запрос и покажет ошибку в консоли.

Access to fetch at 'http://localhost:5000/api/memes'

from origin 'http://localhost:5500' has been blocked by CORS policy

CORS (Cross-Origin Resource Sharing) — это механизм безопасности браузера. Исправляется на стороне сервера: нужно явно разрешить запросы с нашего фронтенда. Делается одной строкой в **Program.cs**.

Что такое fetch

fetch — встроенная функция JavaScript для отправки HTTP-запросов:

```
// GET-запрос — получить данные
const response = await fetch("http://localhost:5000/api/memes");
const data = await response.json();

// POST-запрос — отправить данные
const response = await fetch("http://localhost:5000/api/memes",
{
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Мой мем", category:
"программирование" }),
});

// DELETE-запрос — удалить
const response = await fetch("http://localhost:5000/api/memes/
1", {
  method: "DELETE",
});
```

fetch возвращает **Promise** — обещание, что данные придут позже. Поэтому используем **await**, чтобы дождаться ответа.

Шаг 1. Создание проекта

1.1. Создание рабочей директории

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы и папку для скриншотов:

```
mkdir Lab35_MemeBoard
```

```
cd Lab35_MemeBoard
```

```
mkdir img frontend
```

```
mkdir frontend/css
```

```
mkdir frontend/js
```

1.2. Создание проекта Web API

1. Создайте шаблон **webapi**:

```
dotnet new webapi -n MemesApi
```

cd MemesApi/

mkdir Models Data Controllers

2. Откройте проект в VS Code:

cd ..

code .

3. Откройте терминал и выполните:

cd MemesApi

dotnet restore

1.3. Первый запуск и знакомство со Swagger

1. Установим Swagger, введите в терминале:

dotnet add package Swashbuckle.AspNetCore

dotnet restore

2. Обновите файл **Program.cs**

```
var builder = WebApplication.CreateBuilder(args);

// Регистрация сервисов
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Middleware
if (app.Environment.IsDevelopment()) {
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

1.4. Инициализация Git-репозитория

1. Выполните в терминале:

cd ..

git init -b main

dotnet new gitignore

touch .editorconfig

2. Добавьте в ваш `.editorconfig` файл следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

3. Далее в терминале:

touch README.md

git add .

git commit -m "Initial commit: MemeBoard project"

4. Создайте пустой **public-репозиторий** на GitHub с именем: **Lab35_MemeBoard**

Не добавляйте README, .gitignore, License

5. Привяжите и отправьте:

git remote add origin <URL_репозитория>

git push -u origin main

6. Сделайте **скриншот терминала** с выполненными командами (commit, remote add, push) и сохраните в папку `img` с именем: **gitPushLab35_FIO.png**

Выполните в терминале:

git add .

git commit -m "Added push screenshot"

git push

Шаг 2. Backend — API для мемов

2.1. Создание модели данных

Модель — это описание того, как выглядит наш объект (мем) в коде и в базе данных.

1. В папке **Models** создайте файл **Meme.cs**

2. Пропишите пространство имен:

```
namespace MemesApi.Models;
```

Это адрес нашего файла в системе

3. Объявите класс **Meme**:

```
public class Meme {}
```

Внутри класса **Meme** добавьте свойства, которые описывают мем:

- **Id** — уникальный идентификатор.

```
public int Id { get; set; }
```

- **Title** — текст или название мема. По умолчанию задайте пустую строку `string.Empty`.

```
public string Title { get; set; } = string.Empty;
```

- **Category** — категория (например, «учёба» или «программирование»).

```
public string Category { get; set; } = string.Empty;
```

- **Rating** — оценка мема, по умолчанию 1.

```
public int Rating { get; set; } = 1;
```

- **AddedAt** — дата добавления. Используйте `DateTime.UtcNow`.

```
public DateTime AddedAt { get; set; } = DateTime.UtcNow;
```

2.2. Создание хранилища (In-Memory Data)

Мы с вами храним список мемов в оперативной памяти сервера в виде статического списка.

1. В папке **Data** создайте файл **MemestoStore.cs**

2. Подключите модель:

```
using MemesApi.Models;  
namespace MemesApi.Data;
```

3. Объявите статический класс **MemestoStore**:

```
public static class MemestoStore {}
```

4. **Внутри класса** создайте поле для **ID**:

```
private static int _nextId = 4;
```

Это счетчик для новых записей.

5. **Внутри класса** объявите список мемов:

```
public static List<Meme> Memes { get; } = new() { };
```

6. **Наполните список** тремя объектами `new Meme { ... }`, заполнив все поля (`Id`, `Title`, `Category`, `Rating`, `AddedAt`):

```
new Meme { Id = 1, Title = "Когда код заработал с первого раза",
    Category = "программирование", Rating = 5,
    AddedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc) },
new Meme { Id = 2, Title = "Я и дедлайн в 23:59",
    Category = "учёба", Rating = 4,
    AddedAt = new DateTime(2026, 1, 2, 0, 0, 0, DateTimeKind.Utc) },
new Meme { Id = 3, Title = "Пятница после пар",
    Category = "жизнь", Rating = 5,
    AddedAt = new DateTime(2026, 1, 3, 0, 0, 0, DateTimeKind.Utc) }
```

7. **Внутри класса** добавьте метод для генерации ID:

```
public static int NextId() => _nextId++;
```

Метод `NextId()` возвращает текущее значение `_nextId` и увеличивать его на 1 (инкремент ++).

2.3. Создание контроллера (Логика API)

Контроллер — это «диспетчер», который принимает запросы от фронтенда, обращается к данным и возвращает ответ.

Создайте **Controllers/MemesController.cs**

1. Подготовка каркаса

- Подключите пространства имен:

```
using Microsoft.AspNetCore.Mvc;
using MemesApi.Models;
using MemesApi.Data;

namespace MemesApi.Controllers;
```

Вам понадобятся `Microsoft.AspNetCore.Mvc`, а также ваши собственные `MemesApi.Models` и `MemesApi.Data`.

- Объявите класс **MemesController**. Он должен наследоваться от **ControllerBase**:

```
public class MemesController : ControllerBase { }
```

- Добавьте атрибуты:

```
[ApiController]
[Route("api/[controller]")] ]+
public class MemesController :
```

- `[ApiController]` — сообщает серверу, что это API-контроллер.

- `[Route("api/[controller]")]` — задаёт путь. По умолчанию `[controller]` заменится на название класса без слова `Controller`, то есть адрес будет `api/memes`.

Далее пишем внутри класса!

2. Метод получения данных (GET)

- Реализуйте метод `GetAll()`: Он должен возвращать `ActionResult` со списком мемов. Используйте `Ok(MemesStore.Memes)`.

```
[HttpGet]
public ActionResult<List<Meme>> GetAll() {
    return Ok(MemesStore.Memes);
}
```

- Реализуйте метод `GetById(int id)`:

```
[HttpGet("{id}")]
public ActionResult<Meme> GetById(int id) {
    var meme = MemesStore.Memes.FirstOrDefault(m => m.Id == id);
    if (meme is null) {
        return NotFound(new { message = $"Мем с id={id} не найден" });
    }
    return Ok(meme);
}
```

- Используйте LINQ-метод `FirstOrDefault`, чтобы найти мем по ID.
- Важно: Обязательно добавьте проверку на `null`. Если мема нет, верните `NotFound` с понятным сообщением об ошибке. `if (meme is null)` - это современный стиль C# (pattern matching), он выглядит чище, чем `== null`

3. Метод создания (POST)

Здесь мы принимаем данные от пользователя.

```
[HttpPost]
public ActionResult<Meme> Create([FromBody] Meme meme) {
    if (string.IsNullOrWhiteSpace(meme.Title)) {
        return BadRequest(new { message = "Название мема не может быть пустым" });
    }
    if (meme.Rating < 1 || meme.Rating > 5) {
        return BadRequest(new { message = "Рейтинг должен быть от 1 до 5" });
    }
    meme.Id = MemesStore.NextId();
    meme.AddedAt = DateTime.UtcNow;
    MemesStore.Memes.Add(meme);
    return CreatedAtAction(nameof(GetById), new { id = meme.Id }, meme);
}
```

- Атрибут **[HttpPost]**: Указывает, что метод создает ресурс.
- Валидация:
 - Проверьте, что Title не пустой (используйте `string.IsNullOrEmpty`).
 - Проверьте Rating (он должен быть в диапазоне от 1 до 5).
 - Если данные неверны, возвращайте `BadRequest`.
- Наполнение: Перед сохранением присвойте мему новый ID через `MemesStore.NextId()` и актуальную дату `DateTime.UtcNow`.
- Результат: Вместо обычного `Ok` профессиональнее возвращать `CreatedAtAction` — это вернёт статус 201 и ссылку на созданный объект.

4. Метод удаления (DELETE)

```
[HttpDelete("{id}")]
public ActionResult Delete(int id) {
    var meme = MemesStore.Memes.FirstOrDefault(m => m.Id == id);
    if (meme is null) {
        return NotFound(new { message = $"Мем с id={id} не найден" });
    }
    MemesStore.Memes.Remove(meme);
    return NoContent();
}
```

- Атрибут `[HttpDelete("{id}")]`: Мы передаем ID удаляемого объекта в URL.
- Логика: Сначала найдите мем, как в методе `GetById`. Если нашли — удалите его из списка `MemesStore.Memes.Remove(meme)`.
- Ответ: При успешном удалении принято возвращать `NoContent()` (статус 204).

2.4. Настройка Program.cs и CORS

Чтобы фронтенд смог «достучаться» до бэкенда, нам нужно снять стандартные ограничения безопасности браузера и настроить формат передачи данных.

Откройте **Program.cs** и добавьте.

1. Подключите пространство имён для работы с **json**:

```
using System.Text.Json;
```

2. Формат JSON (CamelCase). По умолчанию C# использует `PascalCase` для свойств (например, `Id`, `Title`), но в JavaScript принято использовать `camelCase` (`id`, `title`). Чтобы они понимали друг друга, измените регистрацию контроллеров:

- Найдите строку **`builder.Services.AddControllers()`**.

- Добавьте к ней `.AddJsonOptions(...)`, чтобы задать политику именования `JsonNamingPolicy.CamelCase`.

```
builder.Services.AddControllers()  
    .AddJsonOptions(options => {  
        options.JsonSerializerOptions.PropertyNamingPolicy =  
        JsonNamingPolicy.CamelCase;  
    });
```

3. Регистрация политики CORS (Сервисы) В блоке `builder.Services` (до того как будет вызван `builder.Build()`) добавьте регистрацию политики:

```
builder.Services.AddCors(options => {  
    options.AddPolicy("AllowAll", policy => {  
        policy.AllowAnyOrigin()  
        policy.AllowAnyMethod()  
        policy.AllowAnyHeader();  
    });  
});
```

- Используйте метод `AddCors`.
- Создайте политику с именем `"AllowAll"`.
- Внутри политики разрешите любой источник (`AllowAnyOrigin`), любой метод (`AllowAnyMethod`) и любые заголовки (`AllowAnyHeader`).

Важно: Это небезопасно для реальных проектов, но идеально подходит для обучения.

4. Активация CORS (Middleware) Мало просто создать политику, её нужно включить в конвейер обработки запросов.

- Добавьте `app.UseCors("AllowAll");`.

```
app.UseHttpsRedirection();  
app.UseCors("AllowAll"); +  
app.UseAuthorization();
```

- **Строгий порядок:** Эта строка должна стоять **после** `app.UseRouting()`, но **обязательно до** `app.MapControllers()`.

Практическое задание

Прежде чем переходить к фронтенду, мы должны убедиться, что бэкенд «отдает» данные.

1. Запустите сервер в терминале:

cd MemesApi/

dotnet run

2. Откройте браузер: **http://localhost:5000/api/memes** — (обратите внимание на порт), должен вернуться JSON с тремя мемами.

3. Откройте Swagger: **http://localhost:5000/swagger** — проверьте все маршруты.

4. Сделайте скриншот **Swagger** → **img/step2_swaggerLab35_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 2: MemesApi with CORS configured"

git push

Шаг 3. Frontend — HTML и стили

На этом этапе мы подготовим «лицо» нашего приложения. Чтобы ускорить процесс, мы воспользуемся готовым каркасом HTML и стилями CSS, так как эти темы вы уже освоили ранее.

3.1. HTML-страница

В этом файле мы подготовим форму для ввода новых мемов и контейнер, в который JavaScript будет «подставлять» карточки из базы данных.

1. В папке **frontend** создайте файл **index.html**

2. Вставьте следующий код (обратите внимание на подключение двух JS-файлов в конце body) (**Обратите внимание, при вставке возможны проблемы, проверьте попробовав сохранить файл Ctrl+S и посмотрев на ошибки Prettier!**):

```
<!DOCTYPE html>
<html lang="ru">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Доска мемов</title>
<link rel="stylesheet" href="css/style.css" />
</head>
<body>
<header>
<h1>😄 Доска мемов</h1>
<p class="subtitle">Коллекция лучших мемов группы</p>
</header>
<main>
<!-- Форма добавления мема -->
<section class="add-form">
<h2>Добавить мем</h2>
<div class="form-row">
<input type="text" id="inputTitle" placeholder="Название мема" />
<select id="inputCategory">
<option value="программирование">🖥 Программирование</option>
```

```

<option value="учёба">📖 Учёба</option>
<option value="жизнь">😄 Жизнь</option>
</select>
<select id="inputRating">
<option value="1">★ 1</option>
<option value="2">★★ 2</option>
<option value="3">★★★ 3</option>
<option value="4">★★★★ 4</option>
<option value="5" selected>★★★★★ 5</option>
</select>
<button id="btnAdd">Добавить</button>
</div>
<p class="error-message" id="errorMessage"></p>
</section>
<!-- Статус загрузки -->
<p class="loading-text" id="loadingText">Загружаем мемы ... </p>
<!-- Сетка мемов -->
<section class="memes-grid" id="memesGrid"></section>
</main>
<script src="js/api.js"></script>
<script src="js/app.js"></script>
</body>
</html>

```

3.2. CSS

Чтобы страница выглядела современно (в стиле карточек), добавим стили.

1. В папке **frontend/css** создайте файл **style.css**
2. Используйте CSS-сетку (Grid Layout) для автоматического расположения карточек.

```

/* Общие стили */
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}
body {
font-family: Arial, sans-serif;
background: #f0f2f5;
color: #1a1a1a;
min-height: 100vh;
}
/* Шапка */
header {
background: #1a1a2e;
color: white;
text-align: center;
padding: 2rem;
}
header h1 {
font-size: 2rem;
margin-bottom: 0.5rem;
}
.subtitle {
color: #a0aec0;
font-size: 0.95rem;
}
/* Основной контент */
main {
max-width: 1100px;
margin: 0 auto;
padding: 2rem 1rem;
}
/* Форма добавления */
.add-form {
background: white;
border-radius: 12px;
padding: 1.5rem;
margin-bottom: 2rem;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}
.add-form h2 {
font-size: 1.1rem;

```

```

margin-bottom: 1rem;
color: #444;
}
.form-row {
display: flex;
gap: 0.75rem;
flex-wrap: wrap;
}
.form-row input,
.form-row select {
padding: 0.6rem 0.9rem;
border: 1px solid #ddd;
border-radius: 8px;
font-size: 0.95rem;
outline: none;
transition: border-color 0.2s;
}
.form-row input {
flex: 1;
min-width: 200px;
}
.form-row input:focus,
.form-row select:focus {
border-color: #667eea;
}
.form-row button {
padding: 0.6rem 1.5rem;
background: #667eea;
color: white;
border: none;
border-radius: 8px;
font-size: 0.95rem;
cursor: pointer;
transition: background 0.2s;
}
.form-row button:hover {
background: #5a6fd6;
}
.form-row button:disabled {
background: #a0aec0;
cursor: not-allowed;
}
.error-message {
color: #e53e3e;
font-size: 0.85rem;
margin-top: 0.5rem;
min-height: 1.2rem;
}
/* Текст загрузки */
.loading-text {
text-align: center;
color: #718096;
padding: 2rem;
font-size: 1rem;
}
/* Сетка мемов */
.memes-grid {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
gap: 1.25rem;
}
/* Карточка мема */
.meme-card {
background: white;
border-radius: 12px;
padding: 1.25rem;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
display: flex;
flex-direction: column;
gap: 0.6rem;
transition:
transform 0.15s,
box-shadow 0.15s;
}
.meme-card:hover {
transform: translateY(-3px);
box-shadow: 0 6px 16px rgba(0, 0, 0, 0.12);
}

```

```

.meme-title {
font-size: 1rem;
font-weight: bold;
color: #1a1a1a;
}
.meme-category {
display: inline-block;
background: #ebf4ff;
color: #3182ce;
font-size: 0.75rem;
padding: 0.2rem 0.7rem;
border-radius: 20px;
}
.meme-rating {
font-size: 0.9rem;
color: #f6ad55;
}
.meme-date {
font-size: 0.75rem;
color: #a0aec0;
margin-top: auto;
}
.meme-card button {
align-self: flex-end;
background: none;
border: 1px solid #fed7d7;
color: #e53e3e;
padding: 0.3rem 0.8rem;
border-radius: 6px;
font-size: 0.8rem;
cursor: pointer;
transition: background 0.15s;
}
.meme-card button:hover {
background: #fff5f5;
}
/* Пустое состояние */
.empty-text {
text-align: center;
color: #a0aec0;
padding: 3rem;
grid-column: 1 / -1;
font-size: 1rem;
}

```

Практическое задание

Откройте **frontend/index.html** через **Live Server** в VS Code. Страница должна показать форму и текст «Загружаем мемы...» — JavaScript мы напишем в следующем шаге.

Выполните в терминале:

git add .

git commit -m "Step 3: frontend HTML and CSS"

git push

Шаг 4. JavaScript — создание API-модуля

Хорошая практика в разработке — выносить сетевые запросы в отдельный файл (сервисный слой). Если в будущем изменится адрес сервера или формат данных, нам не придётся искать запросы по всему проекту.

4.1. Создание сервиса запросов

1. В папке **frontend/js** создайте файл **api.js**
2. Объявите базовый адрес (**СВЕРЬТЕСЬ С API ВАШЕГО СЕРВЕРА**):

```
const API_URL = "http://localhost:5000/api/memes";
```

Адрес нашего API — если поменяется порт, меняем только здесь

3. Реализуйте функцию **getAllMemes()** для получения всех мемов:

```
async function getAllMemes() {  
  const response = await fetch(API_URL);  
  
  if (!response.ok) {  
    throw new Error("Не удалось загрузить мемы");  
  }  
  
  return response.json();  
}
```

- Используйте `async/await`.
- Проверяйте `response.ok`. Если сервер вернул ошибку, выбрасывайте исключение `throw new Error(...)`.

4. Реализуйте функцию **addMeme(title, category, rating)**, для добавления мемов:

```

async function addMeme(title, category, rating) {
  const response = await fetch(API_URL, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      title: title,
      category: category,
      rating: rating,
    }),
  });

  if (!response.ok) {
    // Читаем сообщение об ошибке от сервера
    const error = await response.json();
    throw new Error(error.message || "Не удалось добавить мем");
  }

  return response.json();
}

```

- В объекте настроек fetch укажите method: 'POST'.
- **Важно:** Обязательно добавьте заголовок 'Content-Type': 'application/json', иначе сервер не поймёт, что вы прислали объект.
- Тело запроса body превратите в строку через JSON.stringify.

5. Реализуйте функцию `deleteMeme(id)`, для удаления мема по id:

```

async function deleteMeme(id) {
  const response = await fetch(`${API_URL}/${id}`, {
    method: "DELETE",
  });

  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.message || "Не удалось удалить мем");
  }
}

```

- Используйте шаблонную строку с обратными кавычками: `${API_URL}/${id}`, чтобы подставить ID в URL.
- Укажите метод 'DELETE'.

Разбор ключевых моментов (Теория)

- **response.ok**: Свойство, которое равно true, если статус ответа находится в диапазоне 200–299. Если сервер вернул 400 (ошибка валидации) или 404 (не найдено), response.ok будет false. DELETE возвращает 204 (No Content), и response.ok для него тоже true
- **await response.json()**: Читает тело ответа и преобразует JSON-текст в объект JavaScript. Это асинхронная операция, так как данные могут приходить по сети частями.
- **throw new Error(...)**: Мы принудительно создаем ошибку. Это позволит нам обработать все исключения в одном месте (в файле app.js) с помощью блока try/catch.
- **Сериализация**: Браузер не может отправить серверу объект JS напрямую. JSON.stringify() «упаковывает» объект в текстовый формат, понятный серверу.

Шаг 5. JavaScript — логика приложения

Теперь мы напишем код, который свяжет элементы на странице (кнопки, поля ввода) с нашими функциями из api.js. Мы будем использовать **DOM-манипуляции**, чтобы динамически менять содержимое страницы без её перезагрузки.

5.1. Управление интерфейсом

Создайте файл **frontend/js/app.js** и реализуйте его по частям:

1. **Поиск элементов**: Сначала найдите все нужные теги (форму, инпуты, сетку мемов) через document.getElementById

```
const memesGrid = document.getElementById("memesGrid");
const loadingText = document.getElementById("loadingText");
const errorMessage = document.getElementById("errorMessage");
const inputTitle = document.getElementById("inputTitle");
const inputCategory = document.getElementById("inputCategory");
const inputRating = document.getElementById("inputRating");
const btnAdd = document.getElementById("btnAdd");
```

2. **Отрисовка (Render)**:

- Создайте функцию renderMemes(memes).
- Если массив пуст — выводите сообщение «Мемов пока нет».
- Если данные есть — используйте .map() и .join(""), чтобы превратить массив объектов в одну большую строку HTML-кода.


```
function renderMemes(memes) {
  // Прячем текст загрузки
  loadingText.style.display = "none";
  if (memes.length === 0) {
    memesGrid.innerHTML = '<p class="empty-text">Мемов пока нет.
Добавьте первый!</p>';
    return;
  }
  // Для каждого мема создаём карточку и склеиваем в одну строку
  HTML
  memesGrid.innerHTML = memes.map((meme) => createCardHTML(meme)).
  join("");
}
```

3. **Создание карточки:** В функции createCardHTML(meme) используйте шаблонные строки.

```
function createCardHTML(meme) {
  // Форматируем дату
  const date = new Date(meme.addedAt).toLocaleDateString("ru-RU");
  // Строим строку со звёздочками рейтинга
  const stars = "★".repeat(meme.rating);
  return `
    <div class="meme-card">
      <p class="meme-title">${meme.title}</p>
      <span class="meme-category">${meme.category}</span>
      <p class="meme-rating">${stars}</p>
      <p class="meme-date">Добавлено: ${date}</p>
      <button onclick="handleDelete(${meme.id})">🗑 Удалить</button>
    </div>
  `;
}
```

- Не забудьте отформатировать дату через .toLocaleDateString("ru-RU").
- Используйте "★".repeat(meme.rating) для визуализации рейтинга и 🗑 для удаления

4. **Обработка событий:**

- **Загрузка:** Функция loadMemes должна вызываться сразу при открытии страницы.

```
async function loadMemes() {
  try {
    const memes = await getAllMemes();
    renderMemes(memes);
  } catch (error) {
    loadingText.style.display = "none";
    memesGrid.innerHTML = `<p class="empty-text">❌
Ошибка: ${error.message}</p>`;
  }
}
```

- **Добавление:** В функции `handleAdd` обязательно делайте `trim()` для названия и блокируйте кнопку `btnAdd.disabled = true`, чтобы избежать повторных нажатий во время запроса.

```
async function handleAdd() {
  const title = inputTitle.value.trim();
  const category = inputCategory.value;
  const rating = parseInt(inputRating.value);
  // Очищаем предыдущую ошибку
  errorMessage.textContent = "";
  // Простая проверка на клиенте
  if (!title) {
    errorMessage.textContent = "Введите название мема";
    inputTitle.focus();
    return;
  }
  // Блокируем кнопку пока идёт запрос
  btnAdd.disabled = true;
  btnAdd.textContent = "Добавляем...";
  try {
    await addMeme(title, category, rating);
    // Очищаем поле ввода
    inputTitle.value = "";
    // Перезагружаем список
    await loadMemes();
  } catch (error) {
    errorMessage.textContent = error.message;
  } finally {
    // Разблокируем кнопку в любом случае
    btnAdd.disabled = false;
    btnAdd.textContent = "Добавить";
  }
}
```

- **Удаление:** Перед вызовом API используйте `confirm()`, чтобы пользователь случайно не удалил мем.

```
async function handleDelete(id) {
  if (!confirm("Удалить этот мем?")) return;
  try {
    await deleteMeme(id);
    await loadMemes();
  } catch (error) {
    alert("Ошибка при удалении: " + error.message);
  }
}
```

5. Обработчики событий:

```
btnAdd.addEventListener("click", handleAdd);
```

6. Enter в поле ввода — тоже добавляет:

```
inputTitle.addEventListener("keydown", function (event) {  
  if (event.key === "Enter") {  
    handleAdd();  
  }  
});
```

7. Запуск:

```
loadMemes();
```

5.2. Разбор ключевых моментов

- **memes.map(...).join('')**: Самый быстрый способ превратить массив данных в HTML-список. map создает массив строк, а join склеивает их, чтобы браузер не выводил лишние запятые между карточками.
- **Блок finally**: Идеальное место для кода, который должен сработать при любом исходе (например, вернуть кнопку «Добавить» в рабочее состояние).
- **Событие keydown**: Мы добавили обработку клавиши **Enter**, чтобы пользователю не обязательно было тянуться мышкой к кнопке.

Практическое задание

Это момент истины — проверка взаимодействия фронтенда с реальным сервером.

1. **Запуск Backend**: Убедитесь, что в терминале запущен проект **MemesApi**.

2. **Запуск Frontend**: Откройте **index.html** через **Live Server**.

3. **Проверка функций**:

- Появились ли 3 стандартных мема?
- Работает ли кнопка «Добавить»?
- Удаляются ли карточки?

4. **Стресс-тест**: Выключите сервер (Ctrl+C в терминале) и попробуйте обновить страницу. Вы должны увидеть обработанную ошибку в интерфейсе.

Сделайте скриншот страницы с несколькими мемами

→ [img/step5_boardLab35_FIO.png](#)

Выполните в терминале:

git add .

git commit -m "Step 5: frontend JS complete"

git push

Шаг 6. Разбор в DevTools (Инструменты разработчика)

Настало время заглянуть «под капот» браузера и увидеть, как именно летают данные между вашим фронтендом и бэкендом.

6.1. Анализ GET-запроса (Получение данных)

1. Откройте **DevTools** (F12 или ПКМ → Исследовать элемент) и перейдите на вкладку **Network** (Сеть).
2. Перезагрузите страницу. В списке появится запрос **memes**.
3. Нажмите на него и изучите вкладки:
 - **Headers**: найдите строку **Access-Control-Allow-Origin: ***. Это тот самый CORS-заголовок, который добавил ваш сервер.
 - **Response**: «сырой» JSON, который пришел от ASP.NET.
 - **Preview**: тот же JSON, но в виде удобного раскрывающегося списка.

6.2. Анализ POST-запроса (Отправка данных)

1. Добавьте новый мем через форму на странице.
2. В списке **Network** появится второй запрос **memes**, но уже с методом **POST**.
 - **Payload**: здесь вы увидите данные, которые ваш JS упаковал через **JSON.stringify**.
 - **Response**: сервер должен вернуть созданный объект уже с присвоенным **id**.

6.3. Анализ DELETE-запроса (Удаление)

1. Удалите любой мем. Найдите запрос с методом **DELETE**.
 - Обратите внимание на **URL**: он теперь заканчивается на **ID** мема (например, **/api/memes/3**).
 - **Status Code**: должен быть **204 No Content**.

Практическое задание

1. Откройте **DevTools** → **Network** и добавьте новый мем
2. Найдите **POST**-запрос и посмотрите что отправили и что получили в ответ
3. Сделайте скриншот вкладки **Network** с видимым **POST**-запросом и ответом
4. Сохраните в **img/step6_networkLab35_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 6: studied Network tab in DevTools"

git push

Самостоятельное задание

Задание 1. Счётчик мемов

Сделайте интерфейс информативнее. Добавьте над сеткой текст, показывающий общее количество элементов (например: «Всего мемов: 5»).

Подсказка: создайте `<p id="memesCount"></p>` в HTML, а в функции `renderMemes` обновляйте его текст: `document.getElementById('memesCount').textContent = "Всего: " + memes.length;`

Задание 2. README.md

Создайте в корне проекта README.md. Опишите в нём:

1. Кто выполнил работу (ФИО, группа).
2. Краткую инструкцию: как запустить бэкенд (dotnet run) и как открыть фронтенд.
3. Обязательная таблица — что изучили:

Концепция	Где используется
<code>fetch(url)</code>	GET-запрос к API
<code>fetch(url, { method, headers, body })</code>	POST и DELETE запросы
<code>response.ok</code>	Проверка успешности ответа
<code>response.json()</code>	Чтение JSON из ответа
<code>JSON.stringify(...)</code>	Объект JS → JSON-строка для отправки
<code>async / await</code>	Ожидание ответа от сервера
<code>try / catch / finally</code>	Обработка ошибок при fetch
CORS	Разрешение запросов между разными портами
AddCors + UseCors	Включение CORS в ASP.NET Core — два шага

Выполните в терминале:

git add .

git commit -m "Added README and completed self-study tasks"

git push

Итоговая таблица: что изучили в лабораторной

Концепция	Описание
fetch(url)	GET-запрос — получить данные с сервера
fetch(url, { method: 'POST', ... })	POST-запрос — отправить данные
fetch(url, { method: 'DELETE' })	DELETE-запрос — удалить данные
response.ok	true если статус 200–299
response.json()	Прочитать JSON из тела ответа
JSON.stringify(...)	Превратить объект в JSON-строку
async / await	Дождаться ответа от сервера
try / catch / finally	Обработка ошибок, гарантированный блок
CORS	Браузер блокирует запросы между разными портами
AddCors + UseCors	Включение CORS — два шага, оба обязательны
API_URL — константа	Адрес сервера в одном месте
api.js — отдельный файл	Все fetch-запросы отдельно от логики страницы
button.disabled = true	Блокировка кнопки во время запроса
DevTools → Network	Смотрим запросы, ответы, заголовки CORS